

Desafio Técnico

CaseCellShop

Nível Pleno | Fullstack

Empresa fictícia. Caso usado exclusivamente para fins de avaliação técnica.

CaseCellShop | Documento do Candidato

Resumo rápido

Formato: perguntas conceituais + desenho técnico + mini-projeto fullstack.

Objetivo: avaliar decisões de arquitetura, clareza de trade-offs, maturidade em APIs, consistência de dados, tratamento de erros, testes e uma implementação executável.

Uso de IA: permitido e encorajado quando fizer sentido. Registre os prompts mais relevantes em um PROMPTS.md no repositório ou no final do documento de resposta.

Entrega: respostas conceituais + repositório GitHub público + README + PROMPTS.md, quando aplicável. Priorize clareza, simplicidade e uma entrega executável.

Stack preferencial: Node.js + TypeScript no back-end e React + TypeScript no front-end, pois essa é a stack mais próxima do nosso dia a dia.

Se preferir usar outra stack: tudo bem. Explique brevemente a escolha no README e garanta que o projeto seja simples de executar.

Serviços de apoio: use recursos locais ou simulados, como dados em memória, banco local, cache/fila local, Docker Compose, LocalStack ou mocks. Não é necessário contratar ou configurar conta real em provedor de nuvem.

Critérios de Avaliação

Importante — Método de Avaliação:

Serão considerados principalmente:

- clareza na análise dos problemas e impactos;
- qualidade dos trade-offs e das decisões técnicas;
- arquitetura incremental, sem depender diretamente do ERP em todo acesso;
- consistência de estoque, idempotência e tratamento de concorrência;
- contrato de API e modelo de erros;
- testes, validação e estratégia de verificação;

- implementação fullstack executável e bem organizada;
- comunicação de limitações, decisões e uso de IA.

Bem-vindo

Bem-vindo(a) ao desafio técnico pleno da CaseCellShop. O objetivo deste teste é avaliar como você entende um problema realista, organiza uma solução técnica, toma decisões com trade-offs e entrega uma pequena implementação que demonstre maturidade prática.

Não esperamos uma solução pronta para produção. Esperamos uma solução coerente, executável, explicada e pequena o suficiente para ser avaliada com clareza.

Contexto do case

Você foi contratado(a) como Desenvolvedor(a) na CaseCellShop, uma empresa varejista focada na venda de capinhas para celular. A empresa está passando por hipercrecimento: o que antes eram milhares de acessos diários na loja virtual, hoje se transformaram em milhões.

Arquitetura atual

Componente	Descrição
ERP Central	monolito que gerencia estoque, faturamento, financeiro e contábil. É o coração da empresa.
Loja Virtual	e-commerce que consome produtos, preços e estoque diretamente do ERP via API REST síncrona.
Banco de Dados	ERP utiliza MySQL. Temos acesso de leitura, mas não podemos alterar rotinas, tabelas ou código interno do ERP.
Infraestrutura	tudo hospedado em datacenter próprio, com intenção de evoluir para serviços mais escaláveis.
Monitoramento	há alertas básicos de performance, mas pouca rastreabilidade por pedido ou por requisição.

Os 3 problemas identificados

01 | Performance da vitrine

A vitrine demora muitos segundos para carregar produtos, frustrando clientes logo no início da jornada.

02 | Consistência de estoque

Vários clientes conseguem comprar o mesmo produto quando o estoque acaba. A empresa está vendendo itens que não possui.

03 | Resiliência do checkout

Ao finalizar a compra, a API do ERP demora para processar o pedido e gerar faturamento. A requisição sofre timeout e o cliente perde a compra.

Objetivo de evolução

A CaseCellShop quer reduzir a dependência direta do ERP nas jornadas críticas da loja, melhorar a experiência do cliente e evitar venda sem estoque. A solução deve ser incremental: pense no que pode ser construído primeiro, com risco controlado, sem reescrever todo o ERP.

Parte 1.A — Perguntas Conceituais

Orientação: responda em texto, tópicos ou diagramas simples. Não precisa escrever código nesta parte.

Pergunta 1 — Diagnóstico e trade-offs

Para cada um dos 3 problemas identificados:

- O que você acredita estar causando o problema?
- Qual é o impacto para o cliente e para o negócio?
- Quais seriam pelo menos 2 caminhos possíveis de solução?
- Quais trade-offs, riscos ou custos cada caminho traz?
- Qual caminho você priorizaria primeiro e por quê?

Pergunta 2 — Arquitetura alvo incremental

Proponha uma arquitetura para evoluir a loja sem depender diretamente do ERP em cada requisição crítica. Na resposta, descreva:

- componentes principais;
- fluxo de dados para produtos, estoque e checkout;
- onde usaria cache, fila, banco próprio da loja, jobs ou workers;
- como lidaria com sincronização entre loja e ERP;
- o que você implementaria primeiro em um plano de 30 a 90 dias.

Pergunta 3 — Estoque, concorrência e idempotência

Imagine que dois clientes tentam comprar a última unidade do mesmo produto ao mesmo tempo.

Explique:

- como sua solução evita venda duplicada;
- se existe reserva de estoque, quando ela é criada e quando expira;
- como o checkout lida com retry, timeout ou duplo clique;
- como usaria idempotência para evitar pedidos duplicados;
- como reconciliaria divergências entre loja e ERP.

Pergunta 4 — Contrato de API e modelo de erros

Defina um contrato para o endpoint principal de checkout, por exemplo POST /checkout ou POST /orders. Inclua:

- payload mínimo de entrada;
- resposta de sucesso;
- resposta para erro de validação;
- resposta para estoque insuficiente;
- resposta para falha temporária do ERP ou processamento assíncrono;
- códigos HTTP, errorCode e mensagem amigável quando fizer sentido;
- como o front-end deveria reagir a cada tipo de erro.

Pergunta 5 — Testes e estratégia de validação

Descreva como você testaria a solução.

Considere:

- testes unitários;
- testes de integração da API;
- testes de contrato entre front-end e back-end;
- cenários de concorrência ou múltiplas tentativas;
- testes de estados do front-end;
- o que você automatizaria agora e o que deixaria documentado como próximo passo.

Pergunta 6 — Uso de IA no desenvolvimento

Se usar IA para apoiar a solução:

- que tipo de prompt você usaria?
- que partes você delegaria ou não delegaria à IA?
- como verificaria se a resposta está correta?
- que riscos existem ao aceitar uma sugestão de IA sem revisão?

Parte 1.B — Mini-projeto de Código

Orientação: suba o código em um repositório público pessoal no GitHub e cole o link no final da sua resposta.

Escopo da mini-tarefa

Implemente um pequeno fluxo fullstack de checkout para compra de capinhas de celular.

A aplicação deve permitir que um usuário visualize produtos, escolha uma quantidade e tente finalizar uma compra. O sistema deve lidar com sucesso, erro de validação, estoque insuficiente, tentativa duplicada e falha temporária de processamento.

Você pode usar dados em memória ou banco local. Não é necessário usar autenticação, pagamento real, deploy, cloud real ou layout elaborado.

O que esperamos observar na entrega

Back-end

- ☐ Existe uma API para listar ou consultar produtos.
- ☐ Existe uma API para criar uma tentativa de compra.
- ☐ A API valida entradas inválidas.
- ☐ A API diferencia sucesso, erro de validação, estoque insuficiente e falha técnica.
- ☐ A solução evita vender mais unidades do que o estoque disponível.
- ☐ Há alguma estratégia simples contra pedido duplicado?
- ☐ Há uma simulação clara de processamento lento ou instável do ERP.

Front-end

- ☐ Existe uma tela simples para listar produtos ou selecionar um produto.
- ☐ O usuário consegue informar quantidade e iniciar a compra.
- ☐ A interface mostra loading e evita múltiplos cliques durante o processamento.
- ☐ A interface exibe mensagens compreensíveis para sucesso, estoque insuficiente, entrada inválida e falha temporária.
- ☐ O estado da tela permanece coerente após erro ou retry.

Qualidade e entrega

- ☐ O projeto possui README explicando como instalar e rodar.
- ☐ O README explica decisões técnicas, limitações e próximos passos.
- ☐ A escolha de stack é explicada quando diferente da preferencial.
- ☐ Há testes automatizados ou uma estratégia clara de verificação.
- ☐ O código está organizado de forma compreensível.
- ☐ Bônus: incluir diagrama simples, logs estruturados, endpoint de status do pedido ou teste de concorrência.
- ☐ PROMPTS.md registra os prompts relevantes, quando aplicável.

Limites do escopo

Não esperamos:

- pagamento real;
- autenticação completa;
- integração real com ERP;
- uso obrigatório de Docker;
- deploy em cloud;

- mensageria real em produção;
- layout sofisticado.

Como entregar

- ☐ Respostas das 6 perguntas conceituais (Parte 1.A).
- ☐ Link do repositório GitHub público (Parte 1.B).
- ☐ README com instruções de execução e decisões técnicas.
- ☐ PROMPTS.md, quando aplicável.